

Desktop vs. Web Client Comparison

Remote File Folder Client Project

(a simplified Google Drive analogue)

Yana Utochkina

Stage: Comparative Analysis of the Two Client Approaches
September 30, 2026

Abstract

The **Remote File Folder Client** project ships two clients – a native macOS app (Swift/SwiftUI) and a browser-based app (React/TypeScript) – against one shared Go API and PostgreSQL backend. Both clients already have their own dedicated report describing their internal structure, use cases, and test coverage. This report instead compares the two directly: what they share, where they genuinely differ, and the trade-offs each approach carries.

Contents

1	Introduction	3
2	Shared Elements	3
2.1	One Backend Contract	3
2.2	The Same Core Feature Set	3
2.3	The Same Sync Algorithm	3
2.4	A Persisted Sync Cursor	4
3	Where They Differ	4
3.1	Sync Trigger: Continuous vs. Manual	4
3.2	Folder Access: Native Paths vs. Sandboxed Handles	4
3.3	Sorting and Filtering: Client-Side vs. Server-Side	4
3.4	Local State Storage	5
3.5	Networking Stack	5

3.6 Distribution and Reach	5
3.7 Concurrency Model	5
4 Trade-offs of Each Approach	6
4.1 Desktop (Swift/SwiftUI, native macOS)	6
4.2 Web (React/TypeScript, browser)	6
5 Summary Table	7
6 Conclusion	7

1. Introduction

Both clients present the same personal file workspace, talk to the same API over HTTPS at the same public hostname, and render the same underlying file records. They were not built independently and then compared after the fact – the web client’s folder-sync feature is a direct, deliberate port of the desktop client’s sync engine, adapted to what a browser can actually do. That relationship shapes most of what follows: the two clients agree closely on *what* the product does, and differ mainly in *how* each platform lets them do it.

2. Shared Elements

2.1 One Backend Contract

Both clients speak the exact same wire protocol to the same Go API: user-name/password registration and login with JWT access/refresh tokens, the same file record shape (name, extension, size, created/modified timestamps, uploader, editor), the same upload endpoints (single-shot multipart for small files, a chunked initiate/chunk/complete session for large ones, both capped to stay under the Cloudflare Tunnel body-size limit), and the same `.java/.png` content-preview endpoint. Neither client’s backend contract is a subset or superset of the other’s – they are the same contract, used by two different frontends.

2.2 The Same Core Feature Set

Register/log in, browse files with sortable/filterable attributes, preview `.java` as text and `.png` as an image, upload (via a file picker *and* by dragging files in from the desktop/browser), download, and delete – every one of these exists identically in both clients, differing only in how each platform’s UI toolkit expresses it (SwiftUI views vs. React components).

2.3 The Same Sync Algorithm

Folder synchronization runs the same algorithm on both sides: scan the folder for a flat (non-recursive) manifest of name/size/SHA-256 per file, send it with the last-synced version to `POST /api/sync/diff`, apply the returned download/delete_local/conflict actions, then independently upload new or changed local files and delete the remote copies of locally-removed ones. Both implementations exclude the same three cases from that final pass within a single cycle – a file already flagged as a conflict, a file the server just said to delete, and a file just downloaded this same cycle – since skipping any one of the three lets the pass misread a same-cycle side effect as a genuine local

change. Both resolve a conflict with the same three options (Keep Local, Keep Remote, Keep Both), and both name a "keep both" copy the same way: Name (conflict copy, YYYY-MM-DD).ext.

2.4 A Persisted Sync Cursor

Both clients remember a last-synced version number and a per-file record (remote ID, size, hash) across restarts, so a sync doesn't have to start from nothing every time. The two use different storage technology for it (see §3), but the data they store and why is identical.

3. Where They Differ

3.1 Sync Trigger: Continuous vs. Manual

The desktop client watches the chosen folder in the background via `FSEvents` and additionally polls the server every 45 seconds, so a change on either side is picked up within moments without the user doing anything. A browser cannot watch a local folder persistently in the background at all – so the web client's sync is a single, manual **Sync now** button click: one reconcile pass, on demand, and nothing happens between clicks no matter what changes on either side.

3.2 Folder Access: Native Paths vs. Sandboxed Handles

The desktop client works with ordinary filesystem paths via `FileManager` – once a folder is chosen, it can be read and written freely, indefinitely, with no further permission prompts. The web client can only reach a folder the user explicitly granted through the browser's File System Access API, which returns an opaque, sandboxed handle rather than a path; that handle's permission can lapse between sessions and can only be re-requested from inside a genuine user click, not automatically on page load. The API itself is Chromium-only (Chrome, Edge, Brave) – Firefox and Safari have no equivalent at all, so the entire sync feature is unavailable there, while every other feature (browsing, upload, download, delete) still works in any browser.

3.3 Sorting and Filtering: Client-Side vs. Server-Side

The desktop client fetches its user's full file list once (up to a fixed cap) and sorts/filters it locally in memory on every UI interaction. The web client instead sends the active sort column, order, and extension filter to the server as query parameters and re-fetches a paginated page for every change, never holding more than a virtualized window of rows in memory at once. Both arrive at the

same visible result; they just place the sort/filter work on opposite ends of the connection.

3.4 Local State Storage

The desktop client keeps its sync cursor and known-file entries in a single JSON file under Application Support, read and written through Codable. The web client keeps the same kind of data in IndexedDB, behind a small key-value abstraction, since a browser page has no filesystem of its own to write a plain file into – and stores the chosen folder’s handle there too, which is itself only possible because `FileSystemDirectoryHandle` is structured-cloneable.

3.5 Networking Stack

The desktop client uses `URLSession` for every request, including chunked-upload progress. The web client uses `fetch` for everything except chunked-upload progress reporting, which it does through `XMLHttpRequest` instead, since `fetch` has no upload-progress event at all.

3.6 Distribution and Reach

The desktop client ships as a `.dmg`, needs to be installed once, only runs on macOS, and needs to be reinstalled to pick up a new version. The web client needs no installation at all, runs on any device with a modern browser, and is always the latest version the moment a page loads – at the cost of the sync feature specifically being restricted to Chromium-based browsers, as described above.

3.7 Concurrency Model

Because the desktop client’s sync can be triggered by two independent sources at once – the 45-second poll and a debounced folder-watch callback – its reconcile routine has to guard against running twice concurrently; without that guard, two overlapping passes reading and writing the same state can each work from a stale snapshot of the other’s in-progress changes. The web client has no equivalent risk by construction: a sync pass only ever starts from a single button click, so there is never a second trigger to race against.

4. Trade-offs of Each Approach

4.1 Desktop (Swift/SwiftUI, native macOS)

Advantages:

- Continuous background sync – changes on either side show up without the user doing anything;
- Unrestricted, permission-free access to any local folder once chosen;
- Native OS integration: the Keychain for token storage, native file dialogs, no browser sandbox to work around;
- No browser-compatibility carve-outs – every feature works the same way every time.

Disadvantages:

- macOS-only, and requires installing (and periodically reinstalling) a .dmg;
- Sorting/filtering holds the user’s entire file list in memory and works it over client-side, which is fine at personal-drive scale but doesn’t scale indefinitely;
- The continuous-sync model is architecturally more complex, and that complexity already produced three real concurrency/staleness bugs during development (a folder-switch state leak, a reentrant reconcile race, and a same-cycle download-then-delete) that had to be found and fixed.

4.2 Web (React/TypeScript, browser)

Advantages:

- No installation, runs on any OS with a modern browser, always up to date;
- Server-side sorting/filtering and pagination scale to far larger file counts without ever holding the whole list client-side;
- A strictly simpler sync concurrency model – one button, one pass, no background trigger to race against;
- Inherited an already-debugged sync algorithm, so it launched with all three of the desktop client’s hard-won staleness exclusions already in place, rather than having to rediscover them independently.

Disadvantages:

- No background sync at all – a change made elsewhere is invisible locally until the user remembers to click **Sync now** again;
- The sync feature specifically works only in Chromium-based browsers; it is entirely absent in Firefox and Safari;
- Folder permission is not permanent – it can lapse between sessions and must be re-granted from a fresh user click, adding friction a native app doesn’t have;
- Persisted state lives in browser storage (IndexedDB), which is subject to the browser’s own storage-eviction policy under disk pressure, unlike a native app’s own files on disk.

5. Summary Table

The backend contract, core feature set, and sync algorithm/conflict model are identical for both clients (see §2) and so are left out of the table below – table 1 recaps only the dimensions where the two genuinely differ.

Table 1: Desktop vs. web client, at a glance

Aspect	Desktop (Swift/SwiftUI)	Web (React/TypeScript)
Platform	macOS only	Any OS with a modern browser
Sync trigger	Continuous – FSEvents watch + 45s poll	Manual – one click, one pass
Folder access	Native path, permission-free once chosen	Sandboxed handle; permission can lapse
Sync availability	Always (native app)	Chromium only – absent in Firefox/Safari
Sort & filter	Client-side, over a capped full fetch	Server-side, paginated query params
State storage	JSON sidecar file	IndexedDB, via a key-value abstraction
Networking	URLSession	fetch + XMLHttpRequest
Distribution	Installed .dmg; manual reinstall	No install; always latest on load
Concurrency risk	Needs an explicit reentrancy guard	None by construction

6. Conclusion

Neither client is simply a better version of the other – they trade the same constraint (a browser cannot watch a folder in the background) for two different sets of strengths. The desktop client is where continuous, low-friction folder sync actually belongs, at the cost of being a native, installed, macOS-only app. The web client is where zero-install, cross-platform, always-up-to-date access belongs, at the cost of sync being a manual action gated behind a Chromium-specific browser API. The large area of overlap – the same backend contract, the same core feature set, and, for sync specifically, the exact same algorithm and conflict model – is what makes the two feel like one product with two front doors, rather than two unrelated apps that happen to share a server.